Theoretical Guide
py.Sandu

Élvis Miranda, Leonardo Krauss & Rafael Alencar

1 C++

1.1 System optimization

For faster IO, we can use the following code:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
cout.tie(nullptr);
```

This code disables the synchronization between C++ streams and C streams, which can significantly improve the performance of input and output operations. Additionally, it unties the input and output streams, allowing them to operate independently, further enhancing the speed of IO operations.

1.2 Containers

- **std::vector:** A dynamic array. Most methods give O(1) time complexity for access and O(N) for insertion/deletion (except at the end, which is O(1)).
- **std::list:** A doubly linked list. Most methods give O(1) time complexity for insertion/deletion (given an iterator) and O(N) for access.
- **std::deque:** A double-ended queue. Most methods give O(1) time complexity for access and O(1) for insertion/deletion at both ends, but O(N) for insertion/deletion in the middle.
- **std::stack:** A container adapter that gives the functionality of a stack (LIFO). Most methods give O(1) time complexity for push, pop, and top operations.
- **std::queue:** A container adapter that gives the functionality of a queue (FIFO). Most methods give O(1) time complexity for push, pop, and front/back operations.
- **std::priority_queue:** A container adapter that gives the functionality of a priority queue. Most methods give O(log N) time complexity for push and pop operations, and O(1) for top operation.
- **std::set:** A sorted associative container that contains unique elements. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::map:** A sorted associative container that contains key-value pairs with unique keys. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::unordered_set:** An unordered associative container that contains unique elements. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.
- **std::unordered_map:** An unordered associative container that contains key-value pairs with unique keys. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.

1.3 Algorithms

- **std::sort:** The `std::sort` function in C++ is typically implemented using a hybrid sorting algorithm called Introsort, which combines Quick Sort, Heap Sort, and Insertion Sort. It has an average time complexity of O(N log N).
- **std::lower_bound and std::upper_bound:** These functions are used to find the lower and upper bounds of a value in a sorted range. They have a time complexity of O(log N) since they use binary search.
- **std::next_permutation:** This function generates the next lexicographical permutation of a sequence. It is useful for generating all permutations of a sequence in sorted order. The time complexity is O(N) for each call, where N is the length of the sequence.

2 Number Theory

- **Division Algorithm:** $\forall a, b \in \mathbb{Z}, b \neq 0, \exists q, r \in \mathbb{Z} : a = bq + r, 0 \leq r < |b|$.

- **Inverse:** $\forall a \in \mathbb{Z}, \exists a^{-1} \in \mathbb{Z} : a \cdot a^{-1} \equiv 1 \pmod{m}$.. If m is prime, then $a^{-1} \equiv a^{m-2} \pmod{m}$.
- **Euclide's Lemma:** $\forall a, b, p \in \mathbb{Z}, p$ is prime, $p|ab \Rightarrow p|a$ or $p|b$.
- **Bézout's identity:** $\forall x, y \in \mathbb{Z}^*, \exists a, b \in \mathbb{Z} : ax + by = \gcd(a, b)$.

3 Constants

3.1 Time and memory limits

- **1 second limit:** 1 second is usually enough for a program to perform around 10^8 operations for O(N) algorithms and around 10^6 operations for O(N log N) algorithms.
- **256Mb limit:** 256 megabytes is usually enough to store around 10^7 integers (since each integer typically takes 4 bytes).

3.2 Mathematical

- **Pi (π)**: Usually defined as acos(-1.0), approximately equal to 3.14159. In C++ 20, it is available as `std::numbers::pi`.
- **Euler (e)**: Usually defined as the base of the natural logarithm, approximately equal to 2.71828. In C++ 20, it is available as `std::numbers::e`.
- **Golden Ratio (ϕ)**: Usually defined as (1 + sqrt(5)) / 2, approximately equal to 1.61803. In C++ 20, it is available as `std::numbers::phi`.

3.3 Data type limits

- **Integer limits**: The typical 32 bit signed integer bounds are ($\approx \pm 2 \times 10^9$) and is obtained by using the `INT_MAX` and `INT_MIN` constants, while the 64 bit signed integer bounds are ($\approx \pm 9 \times 10^{18}$) and is obtained by using the `LLONG_MAX` and `LLONG_MIN` constants.
- **Floating-point limits**: The typical 32 bit floating-point bounds are approximately $\pm 3.4 \times 10^{38}$ and is obtained by using the `FLT_MAX` and `FLT_MIN` constants, while the 64 bit floating-point bounds are approximately $\pm 1.7 \times 10^{308}$ and is obtained by using the `DBL_MAX` and `DBL_MIN` constants.

4 Math

5 Notes

6 Identities

7 Geometry

7.1 Lines & Polygons

- **Slope-Intercept:** $y = mx + c$
- **General Line Equation:** $Ax + By + C = 0$
- **Distance from point (x_0, y_0) to line $Ax + By + C = 0$:** $d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$
- **Shoelace Formula (Area of Polygon):** $A = \frac{1}{2} |\sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n)|$
  OBS: All vertices must be in order, either clockwise or counterclockwise
- **Pick's Theorem (Area of Lattice Polygons):** $A = I + \frac{B}{2} - 1$, where I is interior points and B is boundary points

7.2 Circles

- **Area:** πr^2
- **Circumference:** $2\pi r$
- **Equation:** $(x - h)^2 + (y - k)^2 = r^2$

7.3 Basic 2D Geometry

- **Distance Formula:** $\sqrt{\Delta x^2 + \Delta y^2}$
- **Vector Definition:** $\vec{v} = (x, y)$
- **Dot Product ($\vec{a} \cdot \vec{b}$):** $x_1 x_2 + y_1 y_2 = |\vec{a}||\vec{b}| \cos\theta$
  Usage: Finding angles, checking perpendicularity ($\vec{a} \cdot \vec{b} = 0$)
- **Cross Product ($\vec{a} \times \vec{b}$)** :$x_1 y_2 - y_1 x_2 = |\vec{a}||\vec{b}| \sin\theta$
  Usage: 2D orientatiom, calculating signed area, collinearity ($\vec{a} \times \vec{b} = 0$)
- **Midpoint:** $M = (\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2})$

7.4 Algorithms/Concepts

- **Convex Hull (Monotone Chain or Graham Scan):** Finds the smallest convex polygon containging all points
- **Line Segment Intersection:** Using cross products to check if two segments intersect
- **Point in Polygon:** Using ray casting (winding number) algorithm for O(n) complexity or BSP Tree for O(log n) complexity
- **Angle of vector:** Use `atan2(y,x)` to find the angle of a vector.

8 Bitwise

8.1 Techniques

- **Check Even/Odd**: $n\&1$ can be used to check if a number is even (result is 0) or odd (result is 1).
- **Power of 2 check:** $(n\&\&!(n\&(n-1)))$ returns true if n is a power of 2.
- **Count 1s:** To count the number of 1s in the binary representation of a number, you can use the expression $n\&(n-1)$ repeatedly until n becomes 0. Each time you perform this operation, it removes the rightmost 1 from the binary representation, allowing you to count how many times you can do this before n becomes 0.
- **Isolate/Clear Rightmost Set Bit:** To isolate the rightmost set bit of a number, you can use the expression $n\&(-n)$. To clear the rightmost set bit, you can use $n\&(n-1)$.
- **Subset Generation:** Use a loop from 0 to $2^n - 1$ to generate all subsets of a set of size n. Each number in this range represents a subset, where the binary representation indicates which elements are included 1 or excluded 0.

8.2 Algebraic Properties

- **Addition Relationship:** $a + b = (a \oplus b) + 2 \cdot (a \wedge b)$
- **XOR Relationship:** $a|b = (a \oplus b) + (a \wedge b)$
- **AND Relationship:** $a \wedge b = (a + b) - (a \oplus b)$
- **Range XOR:** The XOR of all numbers from 1 to n follows a pattern based on $n \pmod 4$:
  - If $n \mod 4 = 0$, then XOR sum $= n$
  - If $n \mod 4 = 1$, then XOR sum $= 1$
  - If $n \mod 4 = 2$, then XOR sum $= n + 1$
  - If $n \mod 4 = 3$, then XOR sum $= 0$

9 Counting Problems

9.1 Permutation

- **Definition:** A permutation of a set is an arrangement of its members into a sequence or linear order. The number of permutations of n distinct objects is given by $n!$ (n factorial).
- **Permutations with Repetition:** If there are n objects where there are n_1 indistinguishable objects of one type, n_2 indistinguishable objects of another type, and so on, the number of distinct permutations is given by $\frac{n!}{n_1! \cdot n_2! \cdot \ldots}$.
- **Circular Permutations:** The number of ways to arrange n distinct objects in a circle is given by $(n-1)!$, since one object can be fixed to break the circular symmetry.

9.2 Modular Inverse Factorial

1. Calculate $n! \pmod p$ for a given prime p
2. Compute the modular inverse of $n!$ using Fermat's Little Theorem, which states that if p is a prime and a is an integer not divisible by p, then $a^{p-1} \equiv 1 \pmod p$. Therefore, the modular inverse of $n!$ can be calculated as $(n!)^{p-2} \pmod p$.
3. Use the precomputed factorial values and their inverses to efficiently compute the modular inverse of $n!$ for multiple queries.

9.3 Combination

- **Definition:** A combination is a selection of items from a larger set, where the order of selection does not matter. The number of combinations of n distinct objects taken r at a time is given by the formula:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

- **Combinations with Repetition:** If repetition of items is allowed, the number of combinations of n distinct objects taken r at a time is given by the formula:

$$C(n+r-1, r) = \frac{(n+r-1)!}{r!(n-1)!}$$

- **Pascal's Triangle:** The number of combinations can also be found using Pascal's Triangle, where the entry in the n-th row and r-th column corresponds to $C(n, r)$.

10 Progressions

A progression is a sequence of numbers ordered according to a specific, logical rule or formula, where each term is derived from the previous one.

10.1 Sums

Sum of first n natural numbers: $\frac{n(n+1)}{2}$ Sum of squares of first n natural numbers: $\frac{n(n+1)(2n+1)}{6}$ Sum of cubes of first n natural numbers: $\left(\frac{n(n+1)}{2}\right)^2$

Sum of squares of first n odd numbers: $\frac{n(4n^2-1)}{3}$ Sum of squares of first n even numbers: $\frac{2n(n+1)(2n+1)}{3}$

10.2 Odds

General Term: $2n - 1$

Sum: n^2

10.3 Geometric Progression

General Term: $a_1 \times q^{n-1}$

Sum: $\frac{a_1(q^n - 1)}{q - 1}$

Infinite Sum:

$$-1 < q < 1$$
$$\frac{a_1}{1-q}$$

10.4 Even

General Term: $2n$

Sum: $n(n+1)$

10.5 Arithmetic Progression

General Term: $a_1 + (n-1)r$

Sum: $\frac{(a_1 + a_n)n}{2}$